Network Forensics & Performance Analysis Report

1. Executive Summary

This report details the technical analysis of specific network traffic indicators, display filters, and protocol metrics extracted during a network investigation. The primary objective is to identify potential security anomalies (such as cleartext credential exposure and unauthorized port usage) and diagnose transport-layer performance bottlenecks.

The indicators evaluated span across multiple layers of the OSI model, focusing heavily on HTTP, DNS, NetBIOS, and advanced TCP analysis flags. Key findings suggest high-risk exposures if certain criteria (like plaintext keywords) return active frames, alongside actionable metrics for network troubleshooting.

---

2. Methodology & Scope

The analysis evaluates a specific set of Wireshark display filters and protocol field fields to categorize them by threat or functional type, determine analytical confidence, and provide the technical context required for investigation. [1, 2]

Operational Focus Areas

- **Protocol Isolation:** Separating core transport layers (TCP vs. UDP) and filtering administrative noise (DNS/NBNS).
- **Targeted Host Scrutiny:** Isolating conversations matching designated suspect and victim IP profiles.
- **Content Inspection:** Searching packet payloads for sensitive strings.
- **Performance Diagnostics:** Utilizing Wireshark's built-in TCP analysis engine to calculate latency and retransmissions.

4. Key Security & Performance Findings

Critical Security Risk: Cleartext Credential Exposure

The combination of tcp contains "password" and http.request confirms an active posture of hunting for cleartext credentials. If traffic matches these filters, it indicates a structural failure to enforce TLS/SSL encryption (HTTPS). Security personnel must immediately locate the source of these streams to prevent credential harvesting.

Non-Standard Port Activity

Filtering for port 9999 across both TCP and UDP implies the tracking of an atypical service. Malware frameworks, custom reverse shells, and backdoor trojans frequently run on non-standard ports above 1024 to evade baseline monitoring.

Network Congestion Indicators

The diagnostic metrics (tcp.analysis.retransmission and tcp.analysis.zero_window) are critical indicators of transport degradation. High retransmission rates indicate packet drops across network paths, whereas zero window errors isolate the issue directly to an overloaded destination server host unable to process its receive queue. [1, 2]

---

5. Recommended Action Items

1. **Fix Filter Syntax for PCAP Execution:** Use the following unified, syntactically correct filter string to isolate suspicious credential activity on custom ports:

(ip.addr == [VICTIM_IP] and ip.addr == [OUR_IP]) and (tcp.port == 9999 or udp.port == 9999) and (tcp contains "password" or tcp contains "google")

1. **Implement Payload Encryption:** Enforce strict HTTPS/TLS communication across all endpoints communicating with external destinations, specifically blocking outbound HTTP requests that transmit authentication strings.
2. **Conduct Host-Based Audits:** Investigate the endpoints bound to Port 9999 to determine the underlying binary or service listening on that port.

🗓️ **30-Day Incident Remediation Timeline**

This timeline balances urgent containment with long-term infrastructure hardening to minimize disruption.

[Day 1-2: Containment] —> [Day 3-7: Eradication] —> [Day 8-14: Remediation] —> [Day 15-30: Hardening]

Days 1–2: Immediate Containment & Isolation

- **Block Port 9999:** Implement immediate firewall rules (Network and Host-based) to block all inbound and outbound traffic on TCP/UDP port 9999 to isolate the suspected service.
- **Force Credential Reset:** Revoke and force an immediate password reset for any user accounts or API keys associated with the "google" or "password" strings captured in the plaintext streams.
- **Session Termination:** Kill active sessions originating from or terminating at the suspect host IP addresses.

Days 3–7: Eradication & Investigation

- **Identify the Process:** Run host-based audits on the involved endpoints to identify the exact binary, script, or service listening on port 9999.
- **Malware Scans:** Run full Endpoint Detection and Response (EDR) or Antivirus scans on the "victim" and "our ip" hosts to ensure no persistent reverse shells or backdoors exist.
- **Locate Cleartext Sources:** Identify the specific application or legacy internal tool making unencrypted http.request calls containing sensitive parameters.

Days 8–14: Remediation & Configuration Patches

- **Enforce TLS 1.3:** Apply configuration patches to the application server to disable unencrypted HTTP (Port 80) and force HTTPS (Port 443) using modern TLS (Transport Layer Security) protocols.

- **Re-bind Services:** Move legitimate custom services away from port 9999 to standard, monitored enterprise ports, or encapsulate them inside an encrypted VPN tunnel.

Days 15–30: Validation & Continuous Monitoring

- **Run Validation PCAPs:** Capture fresh network traffic samples and re-apply the original Wireshark filters (tcp contains "password") to verify no cleartext credentials remain.
- **SIEM Alerting:** Create automated rules in your SIEM (Security Information and Event Management) system to flag any future usage of port 9999 or cleartext transmission of the keyword "password".

🛠️ **Incident Patch Guide**

Deploy the following configuration patches to secure your environment against these specific findings:

1. Network Firewall Policy Patch (Block Port 9999)

Apply this rule to your perimeter and internal firewalls to drop unauthorized traffic.

- **Action:** DROP / DENY
- **Protocol:** TCP and UDP
- **Source IP:** ANY
- **Destination IP:** ANY
- **Destination Port:** 9999

2. Nginx Web Server Patch (Enforce HTTPS/TLS Redirect)

If the unencrypted web traffic was destined for an internal web server, update its server block configuration to refuse plain text and enforce encryption:

nginx

Redirect all unencrypted HTTP traffic to HTTPS

server {

listen 80;

server_name your-internal-domain.local;

return 301 https://$host$request_uri;

}

Secure HTTPS Server Block

server {

listen 443 ssl;

server_name your-internal-domain.local;

ssl_certificate /etc/ssl/certs/internal_server.crt;

ssl_certificate_key /etc/ssl/private/internal_server.key;

Only allow secure protocols and ciphers

ssl_protocols TLSv1.2 TLSv1.3;

ssl_ciphers HIGH:!aNULL:!MD5;

Rest of your application config...

}

Use code with caution.

3. Host-Based Identification Patch (Linux/Windows)

Run these commands on the affected hosts during your Day 3 investigation to find out what application is using port 9999:

- **For Linux Hosts:**

bash

sudo ss -tulpn | grep 9999

or alternatively

sudo lsof -i :9999

Use code with caution.

- **For Windows Hosts (via PowerShell):**

powershell

Get-NetTCPConnection -LocalPort 9999 | Select-Object LocalAddress, LocalPort, RemoteAddress, RemotePort, State, OwningProcess